

FINITE AUTOMATA



Automata and Their Languages

Martina Seidl

Motivation: Regular Expressions

some text file

```
$ cat test.txt
abcdefghijklmnopqrstuvwxyz
ABCDEFGHIJKLMNOPQRSTUVWXYZ
123AB
1234abc
xxxyyy
zzzaaa
```

some regular expressions

```
$ grep a.*z test.txt
abcdefghijklmnopqrstuvwxyz
```

```
$ grep z*a test.txt
abcdefghijklmnopqrstuvwxyz
1234abc
zzzaaa
```

```
$ grep za test.txt
zzzaaa
```

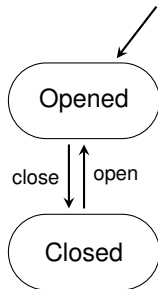
```
$ grep ^a.*z$
abcdefghijklmnopqrstuvwxyz
```

```
$ grep ^b.*z$ test.txt
```

```
$ grep 'ab\|12' test.txt
abcdefghijklmnopqrstuvwxyz
123AB
1234abc
```

Finite Automata (FA)

- very popular mathematical models of computation for defining systems with states, their behavior, and their properties
- virtual device for processing strings
- types of FA:
 - acceptor (accepts or rejects strings)
 - transducer (produces output)
- foundation of other (semi-)formal modeling languages
 - UML state machines
- applications:
 - pattern matching
 - specification of regular languages (for parsers)
 - specification of event streams (actions), e.g., for protocols



FA modeling a door

Formal Languages

Definitions

- An **alphabet** Σ is a finite set of symbols.
- A **word** ω over Σ of size n is a finite sequence $\sigma_1 \dots \sigma_n$ of symbols $\sigma_i \in \Sigma$ (ϵ denotes the empty word, i.e., a sequence of size 0).
- The **set of all words** Σ^* includes all words over Σ .
- A subset L with $L \subseteq \Sigma^*$ is called a **formal language** (over Σ).

Formal Languages

Definitions

- An **alphabet** Σ is a finite set of symbols.
- A **word** ω over Σ of size n is a finite sequence $\sigma_1 \dots \sigma_n$ of symbols $\sigma_i \in \Sigma$ (ϵ denotes the empty word, i.e., a sequence of size 0).
- The **set of all words** Σ^* includes all words over Σ .
- A subset L with $L \subseteq \Sigma^*$ is called a **formal language** (over Σ).

Example

Let $\Sigma = \{a, b\}$. Then $\Sigma^* = \{\epsilon, a, b, aa, ab, ba, bb, \dots\}$. Some formal languages over Σ are:

- | | |
|------------------------|---|
| ■ $L_1 = \{\}$ | ■ $L_3 = \{a, b\}$ |
| ■ $L_2 = \{\epsilon\}$ | ■ $L_4 = \{\epsilon, a, aa, aaa, \dots\}$ |

Definition Finite Automaton (FA)

A **finite automaton** $A = (S, I, \Sigma, T, F)$ consists of:

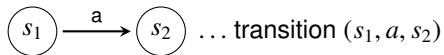
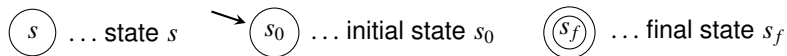
- set of states S
- set of initial states I with $I \subseteq S$
- input alphabet Σ
- transition relation T with $T \subseteq S \times \Sigma \times S$
we also write $s \xrightarrow{a} s'$ for $(s, a, s') \in T$
- set of final states F with $F \subseteq S$

Definition Finite Automaton (FA)

A **finite automaton** $A = (S, I, \Sigma, T, F)$ consists of:

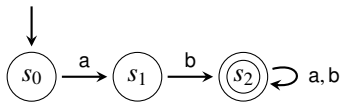
- set of states S
- set of initial states I with $I \subseteq S$
- input alphabet Σ
- transition relation T with $T \subseteq S \times \Sigma \times S$
we also write $s \xrightarrow{a} s'$ for $(s, a, s') \in T$
- set of final states F with $F \subseteq S$

Graphical representation



Example FA

Graphical specification of automaton A :



textual specification of $A = (S, I, \Sigma, T, F)$:

- $S = \{s_0, s_1, s_2\}$
- $I = \{s_0\}$
- $\Sigma = \{a, b\}$
- $T = \{(s_0, a, s_1), (s_1, b, s_2), (s_2, *, s_2)\}$
- $F = \{s_2\}$

Language of FA

Definition

A FA $A = (S, I, \Sigma, T, F)$ **accepts** a word $\omega \in \Sigma^*$ iff there exists $s_i \in S$ and $a_i \in \Sigma$ with

$$s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} \dots \xrightarrow{a_{n-1}} s_{n-1} \xrightarrow{a_n} s_n$$

with $n \geq 0$, $s_0 \in I$, $s_n \in F$ and $\omega = a_1 \dots a_n$.

Language of FA

Definition

A FA $A = (S, I, \Sigma, T, F)$ **accepts** a word $\omega \in \Sigma^*$ iff there exists $s_i \in S$ and $a_i \in \Sigma$ with

$$s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} \dots \xrightarrow{a_{n-1}} s_{n-1} \xrightarrow{a_n} s_n$$

with $n \geq 0$, $s_0 \in I$, $s_n \in F$ and $\omega = a_1 \dots a_n$.

Special case: If $n = 0$, then $\omega = \epsilon$, the empty word.

Language of FA

Definition

A FA $A = (S, I, \Sigma, T, F)$ **accepts** a word $\omega \in \Sigma^*$ iff there exists $s_i \in S$ and $a_i \in \Sigma$ with

$$s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} \dots \xrightarrow{a_{n-1}} s_{n-1} \xrightarrow{a_n} s_n$$

with $n \geq 0$, $s_0 \in I$, $s_n \in F$ and $\omega = a_1 \dots a_n$.

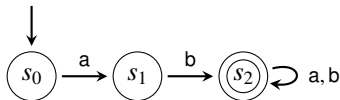
Special case: If $n = 0$, then $\omega = \epsilon$, the empty word.

Definition

The **language** $L(A)$ of A is the set of words accepted by A .

Example Accepted Language

Given automaton A :



$L(A)$ is the sets of all words over alphabet $\{a, b\}$ that start with the string “ab”. Examples:

- $ab \in L(A)$
- $abab \in L(A)$
- $abaaaaa \in L(A)$
- $\epsilon \notin L(A)$
- $baab \notin L(A)$

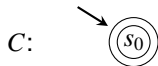
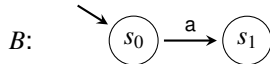
Some Corner Cases

When FA are defined as above, automata can be constructed that

- have no initial states
- have no final states
- accept just the empty word

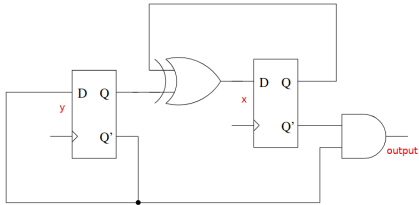
accepted languages:

- $L(A) = L(B) = \emptyset$
- $L(C) = \{\epsilon\}$, $L(A) \neq L(C)$,
 $L(B) \neq L(C)$

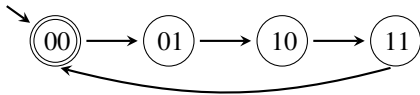


Example: Digital Circuit

Given the following sequential circuit:



Representation as FA:



I/O Automaton

Definition

An **I/O-automaton** $A = (S, i, \Sigma, T, \Theta, O)$ consists of the following components:

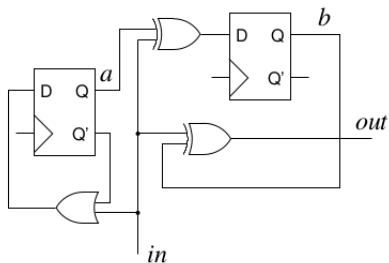
- a (finite) set of states S
- exactly one initial state $i \in S$
- an input alphabet Σ
- a transition function $T: S \times \Sigma \rightarrow S$
- an output alphabet Θ
- an output function $O: S \times \Sigma \rightarrow \Theta$

I/O automata as defined above are also known as Mealy machines

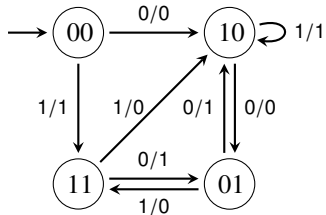
(Moore machines are defined similar, but with output function $O: S \rightarrow \Theta$)

Example: I/O Automaton

Given circuit C :



Representation as I/O automaton:



Behavior of an I/O-Automaton

Let $A = (S, i, \Sigma, T, \Theta, O)$ be an I/O-automaton.

■ The **extended transition function** $\mathcal{T}: S \times \Sigma^* \rightarrow S$ of A is defined as follows:

- $\mathcal{T}(s, \epsilon) = s$
- $\mathcal{T}(s, a\omega) = s''$ iff $\exists s' \in S$ with $s' = T(s, a)$ and $s'' = \mathcal{T}(s', \omega)$ for $a \in \Sigma, \omega \in \Sigma^*$

Behavior of an I/O-Automaton

Let $A = (S, i, \Sigma, T, \Theta, O)$ be an I/O-automaton.

■ The **extended transition function** $\mathcal{T}: S \times \Sigma^* \rightarrow S$ of A is defined as follows:

- $\mathcal{T}(s, \epsilon) = s$
- $\mathcal{T}(s, a\omega) = s''$ iff $\exists s' \in S$ with $s' = T(s, a)$ and $s'' = \mathcal{T}(s', \omega)$ for $a \in \Sigma, \omega \in \Sigma^*$

■ The **extended output function** $O: S \times \Sigma^* \rightarrow \Theta^*$ of A is defined as follows:

- $O(s, \epsilon) = \epsilon$
- $O(s, a\omega) = b\omega'$ with $b = O(s, a), \quad s' = T(s, a), \quad \omega' = O(s', \omega)$

Behavior of an I/O-Automaton

Let $A = (S, i, \Sigma, T, \Theta, O)$ be an I/O-automaton.

■ The **extended transition function** $\mathcal{T}: S \times \Sigma^* \rightarrow S$ of A is defined as follows:

- $\mathcal{T}(s, \epsilon) = s$
- $\mathcal{T}(s, a\omega) = s''$ iff $\exists s' \in S$ with $s' = T(s, a)$ and $s'' = \mathcal{T}(s', \omega)$ for $a \in \Sigma, \omega \in \Sigma^*$

■ The **extended output function** $O: S \times \Sigma^* \rightarrow \Theta^*$ of A is defined as follows:

- $O(s, \epsilon) = \epsilon$
- $O(s, a\omega) = b\omega'$ with $b = O(s, a), \quad s' = T(s, a), \quad \omega' = O(s', \omega)$

The **behavior** $B: \Sigma^* \rightarrow \Theta^*$ of A is defined as $B(\omega) = O(i, \omega)$.

Example: Behavior of I/O Automaton

I/O automaton that reads a sequence of letter a . If the number of a is odd, the output word should end with o , otherwise with e .

Example: Behavior of I/O Automaton

I/O automaton that reads a sequence of letter a . If the number of a is odd, the output word should end with o , otherwise with e .

We specify I/O automaton

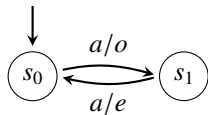
$A = (S, s_0, \Sigma, T, \Theta, O)$ as follows

- $S = \{s_0, s_1\}$

- $\Sigma = \{a\}$

- $\Theta = \{e, o\}$

- definition of T and O :



$$\mathcal{T}(s_0, a^{2n}) = s_0,$$

$$\mathcal{T}(s_0, a^{2n+1}) = s_1,$$

$$\mathcal{T}(s_1, a^{2n}) = s_1,$$

$$\mathcal{T}(s_1, a^{2n+1}) = s_0$$

$$B(a^{2n}) = (oe)^n,$$

$$B(a^{2n+1}) = (oe)^n o$$

I/O Automaton as Finite Automaton

Definition

Let $A = (S, i, \Sigma, T, \Theta, O)$ be an I/O automaton. The FA for A is the automaton $A' = (S, \{i\}, \Sigma \times \Theta, T', S)$ with

$$(s, (a, b), s') \in T' \text{ iff } s' = T(s, a), b = O(s, a)$$

I/O Automaton as Finite Automaton

Definition

Let $A = (S, i, \Sigma, T, \Theta, O)$ be an I/O automaton. The FA for A is the automaton $A' = (S, \{i\}, \Sigma \times \Theta, T', S)$ with

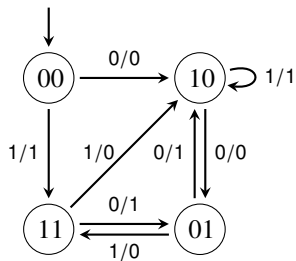
$$(s, (a, b), s') \in T' \text{ iff } s' = T(s, a), b = O(s, a)$$

Proposition

$$B(a_1 \dots a_n) = b_1 \dots b_n \text{ iff } (a_1, b_1) \dots (a_n, b_n) \in L(A')$$

Example: I/O Automaton as Finite Automaton

Given I/O automaton:



Representation as FA automaton:

